

Algorithmique et programmation impérative

Chapitre 4 : Instructions conditionnelles

Hadrien Mélot
Service d'Algorithmique
Département d'Informatique
Faculté des Sciences



Chapitre 4. Instructions conditionnelles

- 1 Expressions booléennes
- 2 Instructions conditionnelles
- 3 Fonctions booléennes
- 4 Entrées au clavier (parenthèse)
- 5 Conversion de réels en entiers (parenthèse)
- 6 Erreurs d'approximation (parenthèse)
- 7 Erreur fréquente
- 8 Exercices interactifs
- 9 Glossaire

Expressions booléennes

Expressions booléennes

Une **expression booléenne** est une expression qui retourne soit vrai, soit faux.

- le type `bool` possède deux valeurs spéciales : `True` (**vrai**) et `False` (**faux**).
- L'opérateur `==` compare deux opérandes et retourne `True` si elles sont égales et `False` sinon.

```
>>> 5 == 5
```

```
True
```

```
>>> 5 == 6
```

```
False
```

Le type bool

Les valeurs `True` et `False` sont de type `bool`, ce ne sont pas des `str`.

```
>>> type(True)
<class 'bool'>
>>> type(False)
<class 'bool'>
>>> type('False')
<class 'str'>
>>> type(2 == 3)
<class 'bool'>
```

Opérateurs de comparaison

L'opérateur `==` est un **opérateur de comparaison**. Il en existe 5 autres qui retournent tous un `bool` :

- $x \neq y$ x n'est **pas égal** à y
- $x > y$ x est **plus grand** que y
- $x < y$ x est **plus petit** que y
- $x \geq y$ x est **plus grand ou égal** à y
- $x \leq y$ x est **plus petit ou égal** à y

Note : `=<` et `=>` ne sont pas acceptés (erreurs de syntaxe)

Comparaison et assignation : ne pas confondre

Une erreur fréquente est de confondre l'opérateur de comparaison `==` et l'opérateur d'assignation `=`

```
>>> x = 2
>>> type(x == 3)
<class 'bool'>
>>> res = (x == 3)
>>> print(res)
False
>>> type(x = 3)
TypeError: type() takes 1 or 3 arguments
>>> res = (x = 3)
SyntaxError: invalid syntax
```

Opérateurs logiques

Il y a 3 opérateurs logiques : `and`, `or` et `not`.

La sémantique de ces opérateurs est similaire à leur signification en anglais : “et”, “ou” et “non”.

On construit des expressions booléennes en utilisant les opérateurs de comparaison et logiques.

Par exemple,

`x > 0 and x < 10`

est vrai si et seulement si `x` est plus grand que 0 **et** plus petit que 10.

Exemples

- x est compris entre 2 et 5

$x \geq 2$ and $x \leq 5$

- x est plus petit que 4 ou plus grand que 10

$x < 4$ or $x > 10$

- n est divisible par 2 ou 3

$n \% 2 == 0$ or $n \% 3 == 0$

- n n'est pas divisible par 4

not ($n \% 4 == 0$)

- x n'est pas plus grand que y

not $x > y$

Comment réécrire les deux dernières expressions sans utiliser `not` ?

Remarque : `not` a une précedence plus faible que les opérateurs de comparaisons

Valeurs non nulles

A priori les opérandes des opérateurs logiques devraient être des `bool` mais Python est moins strict que cela :

```
>>> 17 and True
```

```
True
```

```
>>> 0 and True
```

```
0
```

Toute valeur non nulle est interprétée comme vraie, mais une valeur nulle produit un comportement étrange !

Attention : cette flexibilité peut être pratique dans certains cas mais est source de subtilités et confusion : à éviter (sauf si vous savez ce que vous faites).

Opérateurs de comparaison multiples

Pour tester si n est compris entre 0 et 9, nous pouvons écrire

$n \geq 0$ and $n < 10$

En mathématiques, on écrirait sans doute $0 \leq n < 10$.

En Python, l'expression booléenne suivante est également acceptée :

$0 \leq n < 10$

Cependant, dans la plupart des langages de programmation, utiliser des opérateurs de comparaison multiples est interdit (il faut les composer avec “et”) : cette caractéristique de Python est inhabituelle.

Loi de De Morgan

Les expressions booléennes contenant un `not` appliqué sur une expression composée d'un "et" ou d'un "ou" sont souvent difficiles à comprendre.

```
not (0 < n and n < 1000)
```

"vrai quand il n'est pas vrai que n est plus grand que 0 et n est plus petit que 1000"

La [loi de De Morgan](#) permet de simplifier une expression contenant une négation sur deux termes joints par un "et" ou un "ou" :

`not (A and B)` est équivalent à `(not A) or (not B)`

`not (A or B)` est équivalent à `(not A) and (not B)`

Remarquez comme les "et" et "ou" s'inversent.

Loi de De Morgan

Ainsi,

$\text{not } (n > 0 \text{ and } n < 1000)$

“vrai quand il n’est pas vrai que n est plus grand que 0 et n est plus petit que 1000”

Loi de De Morgan

Ainsi,

`not (n > 0 and n < 1000)`

“vrai quand il n’est pas vrai que n est plus grand que 0 et n est plus petit que 1000”

devient

`not (n > 0) or not (n < 1000)`

Loi de De Morgan

Ainsi,

`not (n > 0 and n < 1000)`

“vrai quand il n’est pas vrai que n est plus grand que 0 et n est plus petit que 1000”

devient

`not (n > 0) or not (n < 1000)`

puis, après simplification :

`n <= 0 or n >= 1000`

“vrai quand n est plus petit ou égal à 0 ou plus grand ou égal à 1000”

Loi de De Morgan

Ainsi,

`not (n <= 0 or n >= 1000)`

“vrai quand il n’est pas vrai que n est plus petit ou égal à 0 et n est plus grand ou égal à 1000”

Loi de De Morgan

Ainsi,

`not (n <= 0 or n >= 1000)`

“vrai quand il n’est pas vrai que n est plus petit ou égal à 0 et n est plus grand ou égal à 1000”

devient

`not (n <= 0) and not (n >= 1000)`

Loi de De Morgan

Ainsi,

`not (n <= 0 or n >= 1000)`

“vrai quand il n’est pas vrai que n est plus petit ou égal à 0 et n est plus grand ou égal à 1000”

devient

`not (n <= 0) and not (n >= 1000)`

puis, après simplification :

`n > 0 and n < 1000`

“vrai quand n est plus grand que 0 et plus petit que 1000”

Instructions conditionnelles

Exécution conditionnelle

Les programmes ont souvent besoin de tester certaines conditions et de modifier leur comportement en fonction de celles-ci.

Les **instructions conditionnelles** permettent de le faire.

Syntaxe

```
if expression booléenne :  
    instructions
```

Comportement

Les instructions (indentées) sont exécutées si l'expression booléenne retourne vrai. Si ce n'est pas le cas, elles ne sont pas exécutées.

L'expression booléenne après le `if` est appelée la **condition**.

Exécution conditionnelle

Exemple

```
>>> x = 2
>>> if x > 0:
    print('x est positif')
```

x est positif

```
>>> if x < 0:
    print('x est negatif')
```

```
>>>
```

- Comme pour le corps des fonctions, les instructions se trouvant dans le corps d'un `if` doivent être indentées
- Il n'y a pas de limite aux nombres d'instructions qui apparaissent dans le corps d'un `if`

L'instruction `pass`

Dans le corps d'un `if` (et d'une fonction), il doit y avoir au moins une instruction.

Occasionnellement, on a besoin (temporairement) d'un corps qui ne contient pas d'instructions (par ex., pour y ajouter du code plus tard et pouvoir se concentrer d'abord sur le reste).

Dans ce cas : on peut utiliser l'instruction `pass`, qui ne fait rien.

```
def f():  
    pass  
  
if x < 0:  
    pass    # TODO: gerer le cas ou x est negatif!
```

Exécution alternative

Une deuxième forme d'instruction `if` est l'**exécution alternative**, dans laquelle il y a deux possibilités : la condition détermine laquelle est exécutée.

Syntaxe

```
if expression booléenne :  
    instructions A  
else:  
    instructions B
```

Comportement

Les instructions `A` ne sont exécutées que si l'expression booléenne (la condition) retourne vrai. Si ce n'est pas le cas, les instructions `B` sont exécutées.

Comme la condition est soit vraie, soit fausse, exactement une des deux alternatives est exécutée. Ces alternatives sont appelées des **branches** car elles permettent de définir différentes branches dans le flot d'exécution.

Exécution alternative

Exemple

```
x = 3

if x % 2 == 0:
    print('x est pair')
else:
    print('x est impair')

x est impair
```

Attention : respectez l'indentation. Il faut apprendre à voir comment la gérer correctement avec IDLE, en mode interactif dans une console ou avec un éditeur de texte en mode script.

Conditions chaînées

Parfois, il y a plus que deux possibilités. Pour créer plus de deux branches on utilise des **conditions chaînées**.

Syntaxe

```
if expression booléenne :  
    instructions A  
elif expression booléenne :  
    instructions B  
...  
else:  
    instructions C
```

Comportement

Les instructions A ne sont exécutées que si la première condition est vraie. Si ce n'est pas le cas, la deuxième condition est testée et ainsi de suite, jusqu'au `else` qui est exécuté ssi toutes les conditions sont fausses.

Conditions chaînées

Exemple

```
if x < y:
    print('x est plus petit que y')
elif x > y:
    print('x est plus grand que y')
else:
    print('x et y sont égaux')
```

- `elif` est une abréviation de “else if”
- il n'y a pas de limite au nombre de `elif`
- s'il y a un `else`, il se trouve à la fin, mais ce n'est pas obligatoire (alors, si toutes les conditions sont fausses, il n'y a rien d'exécuté)

```
if x % 2 == 0:
    print('x est pair')
elif x % 3 == 0:
    print('x est divisible par 3')
```

Conditions imbriquées

Une condition peut être **imbriquée** dans une autre.

```
if x == y:
    print('x et y sont égaux')
else:
    if x < y:
        print('x est plus petit que y')
    else:
        print('x est plus grand que y')
```

La première condition possède deux branches. La première branche contient une instruction seule. La seconde branche contient une autre condition, qui possède également ses deux branches.

Bien que l'indentation montre la structure des conditions imbriquées, elles deviennent vite difficiles à lire. En général, on essaye de les éviter si possible.

Conditions imbriquées

Les opérateurs logiques permettent parfois de simplifier les conditions imbriquées.

```
if x > 0:
    if x < 10:
        print('x is a positive single-digit number')
```

La fonction `print` n'est exécutée que si les deux conditions sont vraies. On peut donc écrire :

```
if x > 0 and x < 10:
    print('x is a positive single-digit number')
```

Valeur de retour et instructions conditionnelles

Pour rappel, l'instruction `return` dans une fonction signifie “termine immédiatement la fonction et utilise l'expression qui suit comme valeur de retour”.

```
def valeur_absolue(x):  
    if x < 0:  
        return -x  
    else:  
        return x
```

Si une fonction qui doit retourner une valeur contient des instructions conditionnelles, il faut s'assurer que toutes les possibilités rencontrent une instruction `return`.

```
>>> def valeur_absolue(x):  
    if x < 0:  
        return -x  
    elif x > 0:  
        return x
```

```
>>> print(valeur_absolue(0))
```

`None`

Valeur de retour et instructions conditionnelles

Une instruction `return` qui n'est pas suivie d'une expression peut être utilisée pour sortir d'une fonction prématurément. Dans ce cas, la valeur spéciale `None` est automatiquement retournée.

```
def aire(r):  
    if r < 0:  
        return  
    return math.pi * r ** 2
```

Fonctions booléennes

Fonctions booléennes

Les fonctions peuvent retourner des booléens, ce qui est souvent très utile pour cacher des tests compliqués dans des conditions ou rendre le code plus clair.

Exemple :

```
>>> def is_divisible(x,y):  
    if x % y == 0:  
        return True  
    else:  
        return False
```

```
>>> is_divisible(6,4)
```

```
False
```

```
>>> is_divisible(6,3)
```

```
True
```

Fonctions booléennes

C'est une bonne habitude de nommer les fonctions booléennes comme des questions dont la réponse est “oui” (True) ou “non” (False) : `est_pair`, `est_premier`, `is_positive`.

Les fonctions booléennes augmentent la lisibilité des instructions conditionnelles.

```
if is_divisible(x,2) and is_positive(x):  
    print('x est un nombre pair positif')
```

Comparer explicitement la valeur de retour d'une fonction booléenne est inutile et alourdit le code.

```
if is_divisible(x,y) == True:  
    print('x est divisible par y')
```

Fonctions booléennes

Comme un opérateur de comparaison retourne une valeur booléenne, on peut écrire `is_divisible` de manière plus concise :

```
def is_divisible(x,y):  
    return x % y == 0
```

Evaluation “lazy”

Lors d'un test `and` ou `or`, Python effectue une évaluation **lazy** :

- `A and B` : si `A` est faux, le test `B` n'est pas évalué, retourne `False`
- `A or B` : si `A` est vrai, le test `B` n'est pas évalué, retourne `True`

Utile si le test `B` est par exemple un appel à une fonction booléenne qui prend du temps à être calculée.

Illustration : script `lazy.py` présenté en auditoire.

Entrées au clavier (parenthèse)

Entrées au clavier

La fonction `input` permet d'obtenir des données entrées au clavier par l'utilisateur.

- quand elle est appelée, le programme s'arrête et attend que l'utilisateur entre quelque chose au clavier
- quand l'utilisateur appuie sur `Return` ou `Enter`, le programme reprend et `input` retourne ce que l'utilisateur a entré sous la forme d'un `str`

```
>>> data = input()  
qu'attends-tu?  
>>> print(data)  
qu'attends-tu?
```

Remarque : en Python 2, on utilise `raw_input` au lieu d'`input`

Entrées au clavier

Avant d'attendre une entrée de l'utilisateur, il est utile de lui préciser ce qu'on attend de lui ! La fonction `input` prend un argument de type `str` pour ce faire.

```
>>> name = input('Quel est votre nom ? ')
```

```
Quel est votre nom ? John Doe
```

```
>>> print(name)
```

```
John Doe
```

Entrées au clavier

Le caractère spécial `\n` peut être utilisé dans un `str` et représente une nouvelle ligne.

```
>>> name = input('Quel est votre nom ?\n')
```

```
Quel est votre nom ?
```

```
John Doe
```

```
>>> print(name)
```

```
John Doe
```

Entrées au clavier

Si vous voulez obtenir un nombre entier ou réel, il faut convertir le `str` retourné par `input`.

```
>>> prompt = 'Quel est votre age ?\n'
>>> data = input(prompt)
Quel est votre age ?
19
>>> print(data, type(data))
19 <class 'str'>
>>> age = int(data)
>>> print('Vous allez bientôt avoir', (age + 1), 'ans')

Vous allez bientôt avoir 20 ans
```

Entrées au clavier

Si l'utilisateur entre quelque chose qui ne peut être correctement converti, on obtient une erreur.

```
>>> prompt = 'Quel est votre taille ?\n'
>>> data = input(prompt)
Quel est votre taille ?
Heu.... en centimetres ou en metres ?
>>> taille = float(data)

ValueError: invalid literal for float()
```

Rappel : Comment appelle-t-on ce type d'erreurs ?

On verra comment gérer cela plus tard.

Interaction avec l'utilisateur : exemple

Problème

Ecrire un programme qui permette de calculer l'aire ou la circonférence d'un cercle de rayon r .

Interaction avec l'utilisateur : exemple

Une solution :

```
import math

def aire(r):
    return math.pi * r ** 2

def circonference(r):
    return math.pi * 2.0 * r

def print_res(res,type):
    print(type, '=', res)
```

Suite slide suivant...

Interaction avec l'utilisateur : exemple

```
prompt = 'Veuillez entrer le rayon du cercle...\n'
rayon = float(input(prompt))

if rayon < 0.0:
    print('Je vais avoir du mal a calculer ca...')
else:
    prompt = 'Voulez-vous connaitre l\'aire ou la circonference ?\n'
    typeCalcul = input(prompt)
    if typeCalcul == 'aire' or typeCalcul == 'Aire':
        res = aire(rayon)
        print_res(res, typeCalcul)
    elif typeCalcul == 'circonference' or typeCalcul == 'Circonference':
        res = circonference(rayon)
        print_res(res, typeCalcul)
    else:
        print('Je ne comprend que \"aire\" ou \"circonference\"')
```

Conversion de réels en entiers (parenthèse)

Conversion de réels en entiers

Pour rappel, si x et y sont deux entiers positifs, le résultat de $x // y$ (division entière) est la partie entière de $\frac{x}{y}$.

D'autre part, la fonction `int` avec un nombre réel x en argument retourne la partie entière de x .

```
>>> 1 // 2
0
>>> int(0.5)
0
```

Attention : en réalité, la division entière calcule un **plancher**¹, alors que `int` **tronque** la partie décimale. Ainsi, avec des nombres négatifs, les comportements sont différents.

```
>>> -1 // 2
-1
>>> int(-0.5)
0
```

1. le plancher de x est le plus grand entier i tel que $i \leq x$.

Conversion de réels en entiers

Sachant que `int` tronque un réel, les fonctions suivantes peuvent être utiles pour convertir un réel en entier.

```
>>> import math
>>> x = 1.34
>>> y = -3.29
>>> z = 2.0
```

- **Plancher** (plancher de x = plus grand entier i tel que $i \leq x$)

```
>>> print(math.floor(x), math.floor(y), math.floor(z))
1 -4 2
```

- **Plafond** (plafond de x = plus petit entier i tel que $i \geq x$)

```
>>> print(math.ceil(x), math.ceil(y), math.floor(z))
2 -3 2
```

- **Arrondi** (entier le plus proche)

```
>>> print(round(x), round(y), round(z))
1 -3 2
```

Remarque : en Python 2, les fonctions d'arrondi retournent des réels et non des entiers, qu'il faut alors convertir en entier.

Conversion de réels en entiers

En ce qui concerne l'arrondi, en Python 3, un nombre positif dont la partie entière est impaire se terminant par 0.5 est arrondi vers le haut. C'est l'inverse si la partie entière est paire.

```
>>> print(round(1.5), round(2.5), round(-1.5), round(-2.5))  
2 2 -2 -2
```

Les choses s'inversent encore si le nombre est négatif.

Voyez-vous pourquoi un “demi” nombre n'est pas toujours arrondi vers le haut (comme c'est le cas en Python 2) ?

Conversion de réels en entiers

En ce qui concerne l'arrondi, en Python 3, un nombre positif dont la partie entière est impaire se terminant par 0.5 est arrondi vers le haut. C'est l'inverse si la partie entière est paire.

```
>>> print(round(1.5), round(2.5), round(-1.5), round(-2.5))  
2 2 -2 -2
```

Les choses s'inversent encore si le nombre est négatif.

Voyez-vous pourquoi un “demi” nombre n'est pas toujours arrondi vers le haut (comme c'est le cas en Python 2) ?

Le but est d'éviter un **biais** s'il y a beaucoup d'arrondis, toujours vers le haut, par exemple dans des applications financières.

Erreurs d'approximation (parenthèse)

Tester l'égalité de deux `float`

Quand on veut tester l'égalité de deux `float`, il faut éviter d'utiliser `==`.

```
>>> x = 0.1
>>> y = (math.sqrt(math.sqrt(x))) ** 4
>>> x == y
False
```

?

Tester l'égalité de deux `float`

Quand on veut tester l'égalité de deux `float`, il faut éviter d'utiliser `==`.

```
>>> x = 0.1
>>> y = (math.sqrt(math.sqrt(x))) ** 4
>>> x == y
False
```

?

Tout calcul impliquant un `float` peut provoquer une **erreur d'approximation**.

Erreurs d'approximation

```
>>> print(x, y)
0.1 0.09999999999999999
```

`float` = erreurs d'approximation

En réalité, même pour `x` il y avait déjà une erreur ! On peut contrôler le nombre de décimales affichées pour un `float` :

```
>>> print(x)
0.1
>>> print('%.3f' % x)
0.100
>>> print('%.50f' % x)
0.100000000000000000555111512312578270211815834045410
>>> print('%.3f' % y)
0.100
>>> print ' %.50f' % y
0.09999999999999999167332731531132594682276248931885
```

Solution

Pour tester l'égalité de deux `float`, il faut vérifier qu'ils sont suffisamment proches, à $\varepsilon = 10^{-9}$ près par exemple.

Ceci peut être écrit grâce à une petite fonction booléenne.

```
>>> def almost_equals(x, y):  
    return abs(x - y) < 10e-9
```

```
>>> x == y
```

```
False
```

```
>>> almost_equals(x, y)
```

```
True
```

Remarque : notez que vous pouvez utiliser une notation scientifique pour les `float`, comme illustré ci-dessus.

Représentation binaire

Certaines erreurs d'approximation proviennent de la manière dont les nombres sont représentés en machine.

Les chiffres utilisés en base 10 vont de 0 à 9, en base 2, nous n'avons besoin que de 0 et de 1 (binaire). Les nombres sont représentés en base 2 sur un ordinateur.

Pourquoi ? Processeurs composés de millions de transistors (imprimés sur un circuit électronique) : chacun ne gère qu'un bit : 0 (le courant ne passe pas) et 1 (le courant passe).

Plus de détails : voir cours *Fonctionnement des ordinateurs*

Représentation binaire

Exemple : le nombre 35 est écrit

- 35 en base 10 car

$$3 \times 10^1 + 5 \times 10^0 = 30 + 5 = 35;$$

- 100011 en base 2 car

$$1 \times 2^5 + 0 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 32 + 2 + 1 = 35.$$

Représentation binaire

Il en va de même pour les fractions.

Exemple : la fraction $\frac{1}{4}$ est écrite

- 0.25 en base 10 car

$$\frac{2}{10^1} + \frac{5}{10^2} = \frac{2}{10} + \frac{5}{100} = \frac{25}{100} = \frac{1}{4};$$

- 0.01 en base 2 car

$$\frac{0}{2^1} + \frac{1}{2^2} = \frac{1}{4}.$$

Erreurs d'approximation

En base 10, la fraction $\frac{1}{3}$ ne peut être représentée de manière exacte, on doit l'approximer car le terme 3 se répète de manière infinie :

0.3333333333333333...

Par contre, la fraction $\frac{1}{10}$ peut être représentée de manière exacte : 0.1

Cependant, en base 2, la fraction $\frac{1}{10}$ est représentée par :

0.000110011001100110011...

où le terme 0011 se répète de manière infinie.

Erreurs d'approximation

Le nombre de bits utilisés par l'ordinateur pour représenter un nombre étant fini explique pourquoi on obtient :

```
>>> print('%%.20f' % 0.1)
0.10000000000000000555
```

Remarques :

- ce n'est donc pas un “bug” de Python (en réalité Python n'y est pour rien)
- l'erreur peut varier d'une machine à l'autre en fonction du nombre de bits alloués pour y représenter un nombre

Erreur fréquente

Effet de bord

Comme expliqué au chapitre précédent, la majorité de vos fonctions seront des fonctions **sans effet de bord**. C'est le cas par exemple de la fonction `circonference` ci-dessous.

```
import math

def circonference(rayon):
    return 2 * math.pi * rayon
```

Mathématiquement, un cercle ne peut pas avoir un rayon négatif. On pourrait donc ajouter un test au début de la fonction pour le vérifier.

Pour simplifier, on va supposer que le rayon est bien un nombre : il reste à tester qu'il ne soit pas négatif.

Comment mettriez-vous cela en place sachant qu'on ne veut pas d'effet de bord ?

Effet de bord

Mauvaise solution :

```
import math

def circonference(rayon):
    if rayon < 0:
        print('Erreur: rayon doit etre >= 0')
    else:
        return 2 * math.pi * rayon
```

Problème : dans certains cas, cette fonction possède maintenant un effet de bord !

```
>>> x = circonference(-10)
Erreur: rayon doit etre >= 0
>>> print(x)
None
```

Effet de bord

Solution sans effet de bord :

- retourner une valeur utilisée comme **code d'erreur** (-1, None, ...)
- préciser dans la documentation ce que signifie ce code d'erreur

```
def circonference(r):  
    """ Retourne la circonference d'un cercle de rayon r  
        r -- rayon du cercle >=0 (retourne None si r < 0)  
    """  
    if r < 0:  
        return None  
    else:  
        return 2 * math.pi * r
```

Avantage : contrôle de la gestion de l'erreur **en dehors** de la fonction

```
x = circonference(-10)  
if x != None:  
    print('x =', x)  
else:  
    print('Entrez un nombre >= 0 pour le rayon')
```

Exercices interactifs

Le dernier théorème de Fermat² affirme qu'il n'y a pas d'entiers positifs a , b et c tels que

$$a^n + b^n = c^n,$$

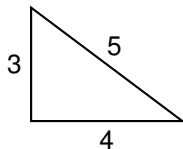
pour n'importe quelle valeur de n plus grande que 2.

Exercice

Ecrivez une fonction booléenne `check_fermat` qui retourne vrai ssi $a^n + b^n = c^n$ et exploitez cette fonction pour faire un programme permettant à l'utilisateur de tenter de trouver un contre-exemple au théorème.

2. Enoncé au 17ième siècle dans une marge et prouvé par Andrew Wiles en 1994.

Avec 3 bâtons, vous pouvez (ou non) les arranger pour former un triangle (non dégénéré). Etant données 3 longueurs, si l'une d'entre elles est strictement plus grande que la somme des deux autres, alors c'est impossible de former un triangle. Sinon, c'est possible.



Exercice

Ecrivez une fonction booléenne `is_triangle` qui, à partir de 3 entiers correspondant à des longueurs, retourne vrai ssi on peut former un triangle avec ces longueurs. Utilisez cette fonction pour faire un programme permettant à l'utilisateur de savoir si 3 longueurs peuvent correspondre à un triangle.

Glossaire

Glossaire

expression booléenne : expression qui retourne soit vrai (True), soit faux (False)

opérateur de comparaison : un des opérateurs qui comparent ses opérandes : ==, !=, <, >, <= et >=.

opérateur logique : un des opérateurs qui combinent des expressions booléennes : and, or et not.

instruction conditionnelle : instruction qui contrôle le flot d'exécution en fonction de certaines conditions

condition : expression booléenne dans une instruction conditionnelle qui détermine quelle branche doit être exécutée

conditions chaînées : instruction conditionnelle avec une série de branches alternatives

conditions imbriquées : instruction conditionnelle qui apparaît à l'intérieur d'une autre instruction conditionnelle